Hidden Markov Model of Four Activity States

Shima Serein

October 2026

Course:

Machine Learning Techniques, Formative 2: Hidden Markov Models

Institution:

African Leadership University

Public GitHub Repository:

https://github.com/supserrr/hidden-markov-models

Data Collection Videos:

https://github.com/supserrr/hidden-markov-models/tree/main/videos

1. Background and Motivation

A phone at the waist, or left on a desk, streams acceleration and rotation. A campus break coach has to tell from that stream which of four states the person is in: still, standing, walking, or jumping. Time with the phone on the desk counts as sedentary, standing in a corridor should not count as a walk across campus, and jumping counts as sport. A phone held flat while standing and a phone lying flat on a table both read about one gravity, so a threshold on the raw magnitude cannot tell them apart. This project treats the four activities as the hidden states of a Gaussian hidden Markov model and windowed sensor features as its observations.

2. Data Collection and Pre-processing

Recording. I made all 200 recordings on 27 October 2026, in one session of about an hour, with Sensor Logger 1.68 on one iPhone 16 Pro, which sampled the accelerometer and the gyroscope every 10 ms (100 Hz). Each activity was recorded as a block of 50 files: standing with the phone held steady at waist level, screen up; walking at a steady pace, holding the phone the same way; continuous jumps; and still, with the phone lying on a flat surface. Every file is 5.6 to 9.9 seconds long, and every activity is well above the 90-second minimum (Table 1).

Activity	Files	Total (s)	Length of one file (s)	Train / test files
Standing	50	438.5	5.58–9.87	40 / 10
Walking	50	393.0	7.21–8.68	40 / 10
Jumping	50	399.0	6.71–9.46	40 / 10
Still	50	438.0	7.65–9.93	40 / 10

Table 1. Recordings per activity. One file is one Sensor Logger export.

Videos. One clip per activity shows the full body and face during collection: standing https://github.com/supserrr/hidden-markov-models/blob/main/videos/standing.mp4, walking https://github.com/supserrr/hidden-markov-models/blob/main/videos/walking.mp4, jumping https://github.com/supserrr/hidden-markov-models/blob/main/videos/jumping.mp4, and still https://github.com/supserrr/hidden-markov-models/blob/main/videos/still.mp4.

Conversion. Sensor Logger exports each recording as a zip. On iOS its accelerometer stream is user acceleration, with gravity already removed, in m/s²; its spread is about 9.8 times that of the

uncalibrated stream in g, which confirms the unit. The gyroscope is in rad/s. src/convert.py puts the gyroscope on the accelerometer timestamps, starts the clock at 0 s, and writes one labelled CSV per recording plus a manifest row. No file has a gap longer than 10 ms or a repeated timestamp.

Sampling rate and windows. There is one phone, so there is one rate to report: 100 Hz. The brief asks for rates to be harmonised across phones, so every file is linearly interpolated onto a 50 Hz grid before any window is cut. A second phone at 50 or 60 Hz would then give the same window length with no other change; for this phone the step is a downsample by two. The window is 1.0 second (50 samples) and the hop is 0.5 second. A walking step takes about half a second and the measured jump rate is about 1.5 per second, so every window holds at least one step or one jump, the native FFT bin is 1 Hz, and the chain gets a new observation twice a second.

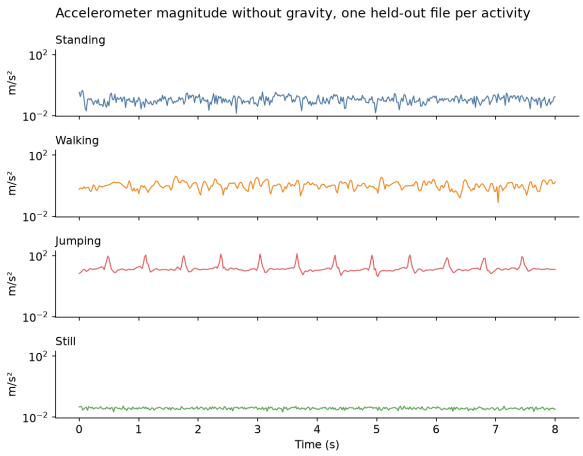


Figure 1. Accelerometer magnitude without gravity for one unseen file per activity, resampled to 50 Hz, on one logarithmic scale. Still sits at the sensor noise floor near 0.01 m/s², standing sways around 0.1, walking moves around 1, and every jump lands as a spike near 100 m/s².

Unseen data. The test set is the last 10 recordings of each activity, by start time. They were made after every training file of the same activity, so no test window sits between two training windows in time. The scaler and the model are fitted on the other 40 files of each activity, which give 2,467 training windows.

Features and normalisation. Each window gives nine features (Table 2). The window's mean acceleration is subtracted first, which removes the tilt of the phone and leaves its motion. Six

features are time-domain and three come from an FFT of the window, zero-padded to eight times its length. The six size features span about seven orders of magnitude on this data: the training mean of spectral energy is 0.0008 for still and 13,454 for jumping. A z-score over that range is set by jumping alone and leaves standing and still closer together than the variance floor of one Gaussian, so the size features go through log10 first. Every feature is then z-scored with the mean and standard deviation of the training windows only, and test windows reuse those values.

Feature	Domain	Why it is in the model (training means)
log acc_mag_mean	Time	Average motion size: 0.009 m/s² still to 13.3 m/s² jumping.
log acc_mag_var	Time	Spread of the magnitude: a landing is a spike, sway is not.
log acc_mag_rms	Time	Energy of the magnitude: separates a hard landing from a step.
log sma	Time	Signal magnitude area: motion on any axis counts.
acc_xy_corr	Time	Horizontal-axis correlation: about 0 at rest, 0.35 to 0.41 moving.
acc_dom_freq	Frequency	Busiest-axis FFT peak; fires in only 27% of walking windows.
log acc_spectral_energy	Frequency	FFT power from 0.15 to 8 Hz: the widest gap between the classes.
log gyro_mag_mean	Time	Rotation: 0.001 rad/s on the desk, 3.3 rad/s jumping.
gyro_dom_freq	Frequency	Gyroscope FFT peak: fires in 86% of jumping windows, near 1.5 Hz.

Table 2. The nine features. "log" marks the six that go through log10 before the z-score.

3. HMM Setup and Implementation

The hidden states Z are standing, walking, jumping, and still. An observation X is the nine-dimensional feature vector of one window. The transition matrix A gives the probability of moving from one state to another between consecutive windows, and the initial vector π gives the probability of the first state. The emission model B is a diagonal Gaussian per state over the scaled features. A diagonal covariance is enough after scaling, and it needs 18 parameters per state instead of 54.

I wrote everything in NumPy (src/hmm.py). Baum-Welch learns π, A, and the Gaussian means and variances. Forward and backward messages stay in log space so a long file cannot underflow. Variances are floored at 10^{-3} and transitions at 10^{-8}, so no log-probability becomes infinite. Learning stops when the relative change in training log-likelihood falls below 10^{-6}. On this data that took 4 iterations, from 9,189.7 to 9,471.1, and the likelihood never fell. A self-test

on a two-state chain with known parameters recovers its path with accuracy 1.00 and checks that the posteriors sum to one.

Parameters start from the labelled training windows: each state's Gaussian is fitted to its own windows, and transition counts get a Dirichlet count of one so an unseen change stays possible. After EM, I matched states to labels using training files only; EM had kept every activity in its own state.

Algorithm choice. Baum-Welch is the learning algorithm because a deployed phone will not have labels, and EM can keep adapting π, A, and B from unlabelled windows. Viterbi is the decoder because sensitivity, specificity, and accuracy count one hard label per window, and the Viterbi path is the single most likely sequence of those labels: the one a live display would show. Forward-backward posteriors drive the E-step and would be the right output if a window were allowed to stay uncertain, but they are not what the table counts.

4. Results and Interpretation

The unseen set is 40 files and 581 windows. The Viterbi path is correct on all 581 windows, and the majority label is correct on all 40 files. In Table 3 a sample is one held-out window. The labelled starting point already scores 1.000 before Baum-Welch, and EM changes no held-out label.

State (activity)	Number of samples	Sensitivity	Specificity	Overall accuracy
Standing	138	1.000	1.000	1.000
Walking	134	1.000	1.000	1.000
Jumping	149	1.000	1.000	1.000
Still	160	1.000	1.000	1.000

Table 3. One-versus-rest sensitivity and specificity on the 40 unseen files. Overall accuracy is the same quantity on every row.

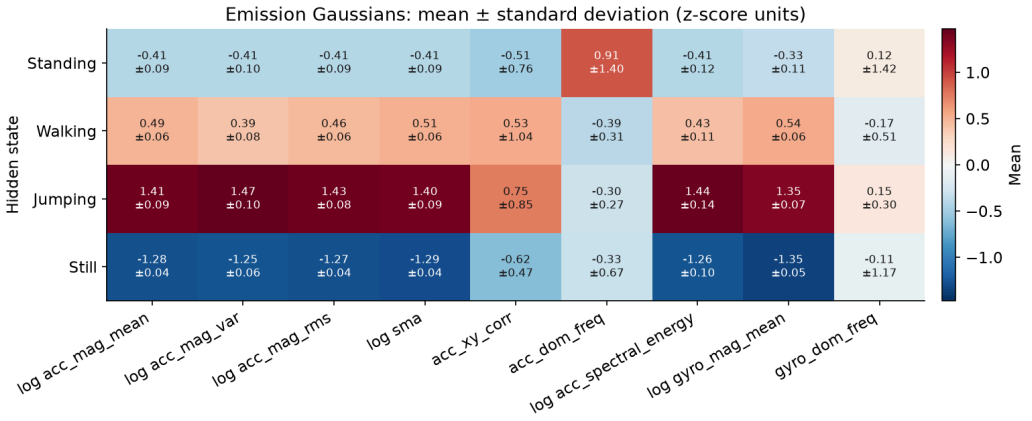


Figure 2. Emission model B: mean and standard deviation of each state's Gaussian on every scaled feature.

On every size feature in Figure 2 the four means are ordered still, standing, walking, jumping, about 0.9 pooled standard deviations apart, while each state's own spread is at most 0.14; that gap explains the exact decoding. The frequency features carry less: their spreads are wide, and the acceleration peak fires in only 27% of walking windows, because a one-second window holds about two strides.

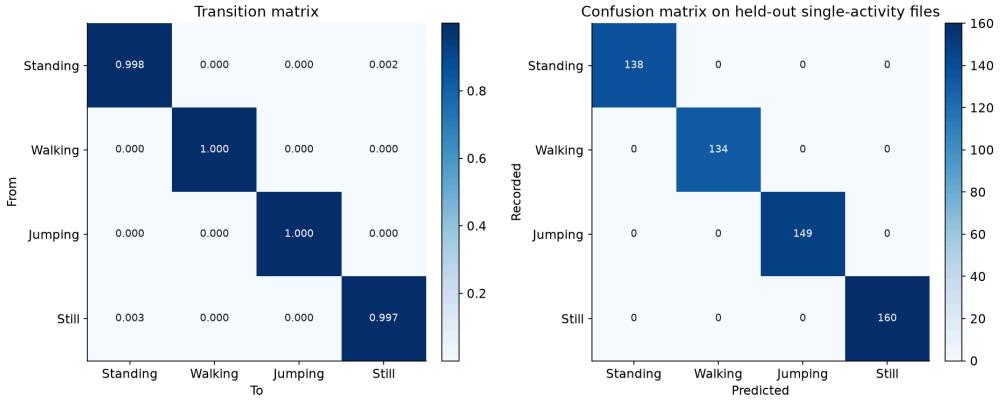


Figure 3. Left: learned transition matrix A; each row sums to 1. Right: confusion matrix on the 581 unseen windows.

The transition matrix is close to the identity. Self-transitions are 0.998 for standing, 0.997 for still, and 1.000 for walking and jumping. With a half-second hop, a self-transition a means an expected stay of 0.5 / (1 − a) seconds: about 5 minutes standing and 2.5 minutes still, which is a plausible length for those states, but an endless walk or jump is not. This comes from the protocol: every file holds one activity, so every pair of consecutive training windows is a

self-transition. The small standing-to-still and still-to-standing mass (0.002 and 0.003) is what EM assigns when a quiet standing window looks partly like still.

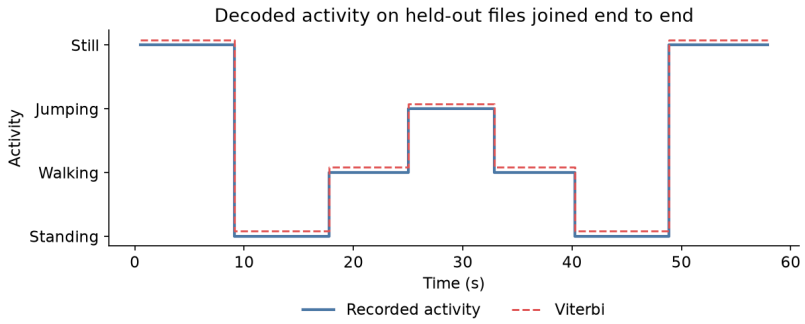


Figure 4. Viterbi on seven unseen files joined end to end: still, standing, walking, jumping, walking, standing, still. The changes are file boundaries, not recorded transitions.

No recording yet changes activity mid-file, so Figure 4 joins seven unseen files into one 58-second sequence. Viterbi follows every change, in both directions, without a wrong window. It switches even though A gives each change a probability near 10^{-8}, because the emissions of neighbouring states differ by far more than that penalty.

5. Discussion and Conclusion

Easiest and hardest activities. Jumping was the easiest state: its spectral energy is about 700 times walking's, and its gyroscope peak fires in 86% of its windows. Walking came next. Standing and still were the hard pair. The phone was flat and screen-up in both, so orientation does not separate them; only the size of the motion does, about 0.1 m/s² of sway against 0.01 m/s² of sensor noise. On a plain z-score that difference is smaller than the variance floor, and Baum-Welch merged the two states: accuracy fell to 0.862, standing sensitivity to 0.587, and the model learned a standing-to-still probability of 0.52 that no recording supports. The log transform prevents this.

Generalisation. On unseen files from the same session the model makes no errors, but the test is narrow. Training and test files share one person, one phone, one way of holding it, and one hour, so the test only asks whether later repetitions look like earlier ones; it says nothing about a pocket, another surface, or another person's gait.

Sensor noise and sampling rate. Sensor noise sets the still state at about 0.01 m/s², ten times below standing sway, so noise caused no errors here. The 100 Hz stream was clean. Downsampling to 50 Hz skips a low-pass filter, so sharp landings can fold energy from above 25 Hz into the band below; a filter before resampling would remove that. The window length mattered more than the rate: one second is too short for a reliable cadence peak, and a two-second window roughly doubles the share of walking windows with a peak, from 27% to 55%, at the cost of a slower response to a change.

Improvements. The most useful next data are recordings that change activity, such as still to standing to walking, labelled at each change, so that A learns real behaviour instead of the protocol. After that come a second participant and phone, a pocket position, the gravity direction as an extra feature, and a Dirichlet prior kept in every M-step so a rare change cannot fall to the 10^{-8} floor.